

Протокол

Автор: Андрій Білий (<https://github.com/bilyilandrii>)

Project GitHub: https://github.com/bilyilandrii/soft_arch_hw_bilyi.git

Programming language + Framework: Python + FastAPI

Коректність та Функціональність

Перевіряю за допомогою функції docs/ від FastAPI. Воно допомагає швидко побачити які ендпоінти доступні та протестити їхню валідність. Код кожного з сервісів можна знайти у відповідній папці.

Спочатку пропоную показати чи працює базово поповнення рахунку користувача та змога переглянути його баланс.

Запит на транзакцію:

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": 100
  }'
```

Відповідь:

Code	Details
200	Response body <pre>{ "transaction_id": "1771438677254461668", "balance": 100 }</pre>  Download

Запит на перегляд балансу:

```
curl -X 'GET' \
  'http://localhost:8080/user/user_test' \
  -H 'accept: application/json'
```

Перевірка

Code	Details
200	Response body <pre>{ "balance": 100, "transactions": [{ "user_id": "user_test", "amount": 100, "transaction_id": "1771438677254461668" }] }</pre>  Download

Як можемо побачити система коректно поповнила баланс користувачеві, повернувши баланс користувача та ідентифікатор транзакції (в моєму випадку це просто timestamp). GET запит повернув актуальний баланс юзера та список всіх його транзакцій (на разі лише одна).

Тепер можна зробити декілька запитів на один і той самий акаунт та перевірити чи всі логи будуть збережені. Будем використовувати того ж user_test

Запити:

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": -50
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": -300
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": 500
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_test",
    "amount": -9
  }'
```

Відповіді:

Code	Details
200	Response body <pre>{ "transaction_id": "1771439990206620054", "balance": 50 }</pre>  Download

Code	Details
200	Response body <pre>{ "transaction_id": "1771440066884545131", "balance": -250 }</pre>  Download

Code	Details
200	Response body <pre>{ "transaction_id": "1771440150505589836", "balance": 250 }</pre>  Download

200	Response body <pre>{ "transaction_id": "1771440321249192596", "balance": 241 }</pre>  Download
-----	---

Перевірка:

Запит такий самий.

Code

Details

200

Response body

```
{
  "balance": 241,
  "transactions": [
    {
      "user_id": "user_test",
      "amount": 100,
      "transaction_id": "1771438677254461668"
    },
    {
      "user_id": "user_test",
      "amount": -50,
      "transaction_id": "1771439990206620054"
    },
    {
      "user_id": "user_test",
      "amount": -300,
      "transaction_id": "1771440066884545131"
    },
    {
      "user_id": "user_test",
      "amount": 500,
      "transaction_id": "1771440150505589836"
    },
    {
      "user_id": "user_test",
      "amount": -9,
      "transaction_id": "1771440321249192596"
    }
  ]
}
```

 Download

 Download

Можна переконаватися що всі логи користувача були зареєстровані з відповідним id для транзакції.

Далі можна зробити запити на декілька акаунтів та перевірити чи відображається баланс усіх користувачів (GET /accounts)

Запити:

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_other1",
    "amount": 10
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_other2",
    "amount": 100
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_other3",
    "amount": 1000
  }'
```

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_other4",
    "amount": -1500
  }'
```

Запит на перегляд балансу усіх користувачів:

```
curl -X 'GET' \
  'http://localhost:8080/accounts' \
  -H 'accept: application/json'
```

Перевірка:

Code	Details
200	Response body <pre>{ "user_test": 241, "user_other1": 10, "user_other2": 100, "user_other3": 1000, "user_other4": -1500 }</pre>  Download

Бачимо що id юзерів та їхні баланси співпадають, отже система коректно змогла опрацювати ці транзакції та вивести баланс кожного з користувачів.

Нижче можете побачити **докер логи** що належать кожному з мікросервісів, що підтверджують вищезгадані запити.

FACADE:

```
facade-1 [FACADE] Received request from user 'user_test' for amount 100
facade-1 [FACADE] Completed transaction 1771438677254461668. Final balance: 100
facade-1 INFO 151.101.64.223:35520 - "POST /process HTTP/1.1" 200
facade-1 INFO 151.101.64.223:37386 - "GET /user/user_test HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_test' for amount -50
facade-1 [FACADE] Completed transaction 1771439990206620054. Final balance: 50
facade-1 INFO 151.101.64.223:27085 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_test' for amount -300
facade-1 [FACADE] Completed transaction 1771440066884545131. Final balance: -250
facade-1 INFO 151.101.64.223:28329 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_test' for amount 500
facade-1 [FACADE] Completed transaction 1771440150505589836. Final balance: 250
facade-1 INFO 151.101.64.223:22779 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_test' for amount -9
facade-1 [FACADE] Completed transaction 1771440321249192596. Final balance: 241
facade-1 INFO 151.101.64.223:45143 - "POST /process HTTP/1.1" 200
facade-1 INFO 151.101.64.223:23813 - "GET /user/user_test HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_other1' for amount 10
facade-1 [FACADE] Completed transaction 1771440783122209921. Final balance: 10
facade-1 INFO 151.101.64.223:64024 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_other2' for amount 100
facade-1 [FACADE] Completed transaction 1771440890135441720. Final balance: 100
facade-1 INFO 151.101.64.223:48329 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_other3' for amount 1000
facade-1 [FACADE] Completed transaction 1771440917040040719. Final balance: 1000
facade-1 INFO 151.101.64.223:16925 - "POST /process HTTP/1.1" 200
facade-1 [FACADE] Received request from user 'user_other4' for amount -1500
facade-1 [FACADE] Completed transaction 1771440947485282469. Final balance: -1500
facade-1 INFO 151.101.64.223:20448 - "POST /process HTTP/1.1" 200
facade-1 INFO 151.101.64.223:63864 - "GET /accounts HTTP/1.1" 200
```

LOGGING:

```
logging_service-1 [LOGGING] Stored transaction 1771438677254461668 for user 'user_test'
logging_service-1 INFO 172.18.0.4:58672 - "POST /transaction HTTP/1.1" 200
logging_service-1 INFO 172.18.0.4:42364 - "GET /transaction/user_test HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771439990206620054 for user 'user_test'
logging_service-1 INFO 172.18.0.4:35702 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440066884545131 for user 'user_test'
logging_service-1 INFO 172.18.0.4:47244 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440150505589836 for user 'user_test'
logging_service-1 INFO 172.18.0.4:40668 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440321249192596 for user 'user_test'
logging_service-1 INFO 172.18.0.4:54700 - "POST /transaction HTTP/1.1" 200
logging_service-1 INFO 172.18.0.4:44182 - "GET /transaction/user_test HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440783122209921 for user 'user_other1'
logging_service-1 INFO 172.18.0.4:36392 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440890135441720 for user 'user_other2'
logging_service-1 INFO 172.18.0.4:50664 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440917040040719 for user 'user_other3'
logging_service-1 INFO 172.18.0.4:53704 - "POST /transaction HTTP/1.1" 200
logging_service-1 [LOGGING] Stored transaction 1771440947485282469 for user 'user_other4'
logging_service-1 INFO 172.18.0.4:43766 - "POST /transaction HTTP/1.1" 200
```

COUNTER:

```
counter_service-1 [COUNTER] User 'user_test' applied 100. New balance: 100
counter_service-1 INFO 172.18.0.4:42986 - "POST /transaction HTTP/1.1" 200
counter_service-1 INFO 172.18.0.4:60594 - "GET /balance/user_test HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_test' applied -50. New balance: 50
counter_service-1 INFO 172.18.0.4:50288 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_test' applied -300. New balance: -250
counter_service-1 INFO 172.18.0.4:41540 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_test' applied 500. New balance: 250
counter_service-1 INFO 172.18.0.4:35092 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_test' applied -9. New balance: 241
counter_service-1 INFO 172.18.0.4:35008 - "POST /transaction HTTP/1.1" 200
counter_service-1 INFO 172.18.0.4:38240 - "GET /balance/user_test HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_other1' applied 10. New balance: 10
counter_service-1 INFO 172.18.0.4:53674 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_other2' applied 100. New balance: 100
counter_service-1 INFO 172.18.0.4:33110 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_other3' applied 1000. New balance: 1000
counter_service-1 INFO 172.18.0.4:35640 - "POST /transaction HTTP/1.1" 200
counter_service-1 [COUNTER] User 'user_other4' applied -1500. New balance: -1500
counter_service-1 INFO 172.18.0.4:34088 - "POST /transaction HTTP/1.1" 200
counter_service-1 INFO 172.18.0.4:44534 - "GET /balance HTTP/1.1" 200
```

Тестування Продуктивності

Імплементація знаходиться у файлі perf_test.py. Під час тестування зайві принти були забрані та програми позакривані. Час для сервісів обраховується за кумулятивним часом, де для кожного запиту я рахую час на віддалений виклик до цих сервісів. Для зчитування та скидання попередніх вимірювань я вирішив додати відповідні HTTP методи: GET /metrics та POST /metrics/reset.

Нижче можете побачити виконання скрипта та його аутпут:

```
=====
Starting Scenario 1 (10 Clients, 10 Unique Accounts)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 173.96 seconds
Requests Per Second (RPS): 574.83

--- Internal Time Contributions ---
Logging Service Total Delay: 1014.24s
Counter Service Total Delay: 991.39s

--- Balance Verification ---
user_0 Balance: 10000 (Expected: 10000)
user_1 Balance: 10000 (Expected: 10000)
user_2 Balance: 10000 (Expected: 10000)
user_3 Balance: 10000 (Expected: 10000)
user_4 Balance: 10000 (Expected: 10000)
user_5 Balance: 10000 (Expected: 10000)
user_6 Balance: 10000 (Expected: 10000)
user_7 Balance: 10000 (Expected: 10000)
user_8 Balance: 10000 (Expected: 10000)
user_9 Balance: 10000 (Expected: 10000)
```

```
=====
Starting Scenario 2 (10 Clients, 1 Shared Account)
=====

--- Performance Results ---
Total Requests: 100000
Total Time: 174.48 seconds
Requests Per Second (RPS): 573.13

--- Internal Time Contributions ---
Logging Service Total Delay: 1011.81s
Counter Service Total Delay: 993.84s

--- Balance Verification ---
Shared Account Balance: 100000 (Expected: 100000)
```

Аналіз Результатів:

Можна одразу побачити що total time та часи окремих сервісів (logging та counter) суттєво відрізняються. Це як і через асинхронність, так і через різні підходи до обрахунку часу, total time просто рахує загалом, скільки часу пройшло (real-world), а для сервісів час **кожного** реквесту акумулюється до однієї змінної. Такий спосіб дозволяє краще побачити наскільки добре справляється сервіс із навантажкою та можна краще побачити різницю між ефективністю підрахування та логування.

Як можете побачити, кількість реквестів на секунду доволі типова для такого сетапу (асинхронний + однопоточний). Для більших цифр можна або змінити логіку роботи сервісів, або додати трішки потоків. Загальний час між двома сценаріями відрізняється не сильно та й за такої структури кількість дій буде виконуватися майже однакова.

Різниця між counting та logging доволі незначна, всього приблизно 20 секунд на плечах першого. Це можна пояснити тим як у Pythonі оновлюється capacity та size хеш мапи. Оскільки запитів досягає аж 100к, то мапа буде для кожного реквесту створювати нову ключ-пару, через що доволі велике використання пам'яті та постійний resizing. Це працює так для обох сценаріїв. З іншого боку, counter service не страдає від цього, оскільки мапа буде налічувати в найгіршому випадку 10 (сценарій 1), а в найкращому 1 (сценарій 2). У свою чергу, математичні обчислення не настільки обтяжливі. Незважаючи на це, головним злодієм такої затримки є комунікація через мережу. Сервіси виконують доволі базові операції, які в теорії виконуються майже миттєво (та й різниця в часі між сервісами незначна), але тут цифри майже космічні. Можу припустити, що більшість часу витрачається на доставку пакетів по мережі та парсинг хедерів. Особливі затримки можуть ставатися, якщо пакет було втрачено по дорозі і прийдеться його відновлювати (TCP).

Висновок:

Загалом, система працює коректно та змогла вистояти наплив 100-чі запитів. Теоретично, вузьким місцем продуктивності є не робота окремих сервісів, а саме спосіб комунікації з ними (мережа). Будем надіятись, що наступні домашні зможуть виправити цю проблему.